

Cahier des charges – Projet VueJS : Plateforme de gestion de parking auto

1. Nom du projet

AutoPark Manager

2. Objectif général

Développer une application web responsive et mobile-first permettant de présenter un parking automobile, afficher les places disponibles, filtrer les catégories, réserver une place, payer en mode sandbox, suivre les ventes/réservations et administrer dynamiquement les contenus depuis un tableau de bord.

Le projet devra être développé avec **VueJS 3**, **Tailwind CSS**, **Pinia**, **Vue Router**, et un backend basé sur **Firebase** ou **Supabase**.

3. Objectifs pédagogiques

Ce projet doit permettre aux stagiaires de pratiquer :

- Vue 3 avec Composition API
 - Vue Router
 - Pinia pour la gestion d'état
 - Lifecycle hooks : `onMounted`, `onUpdated`, `onUnmounted`, `watch`, `watchEffect`
 - Composables réutilisables
 - Authentification utilisateur
 - Rôles : client, admin
 - CRUD complet
 - Backend Firebase ou Supabase
 - Realtime database / subscriptions
 - Upload d'images
 - Formulaires avec validation
 - Pagination, recherche et filtres
 - Panier et réservation
 - Paiement sandbox
 - Dashboard admin
 - Design responsive avec Tailwind CSS
 - Animations UI
 - Structuration professionnelle d'un projet frontend
-

4. Stack technique imposée

Frontend

- VueJS 3
- Vite
- Vue Router
- Pinia
- Tailwind CSS
- Headless UI ou composants personnalisés
- Heroicons / Lucide Icons
- VueUse recommandé
- Vuelidate, vee-validate ou validation maison
- Framer Motion compatible Vue, AutoAnimate, AOS ou transitions Vue natives

Backend au choix

Option 1 : **Supabase**

- Supabase Auth
- Supabase Database PostgreSQL
- Supabase Storage
- Supabase Realtime
- Row Level Security

Option 2 : **Firebase**

- Firebase Auth
- Firestore
- Firebase Storage
- Firebase Realtime updates
- Security Rules

Paieiment sandbox

Au choix :

- Stripe sandbox
- Paystack test mode
- Flutterwave test mode

Le paiement peut être simulé si l'intégration complète prend trop de temps, mais le workflow doit être propre : panier → checkout → paiement → réservation confirmée.

5. Pages publiques à développer

5.1 Landing page

La landing page doit être propre, moderne et responsive.

Sections obligatoires :

1. Header fixe ou sticky
2. Hero section avec call-to-action
3. Présentation du parking
4. Avantages du service
5. Statistiques : places disponibles, réservations, clients satisfaits
6. Catégories de parking
7. Galerie virtuelle
8. Comment ça marche ?
9. Témoignages clients
10. Tarifs ou formules
11. FAQ
12. Formulaire de contact
13. Footer complet

Exigences UI :

- Design mobile-first
- Animation légère sur les sections
- Boutons cohérents
- Couleurs uniformes
- Espacements réguliers
- Typographie propre
- Version mobile, tablette et desktop

5.2 Galerie / parking virtuel

Cette page affiche les places ou zones de stationnement disponibles.

Fonctionnalités obligatoires :

- Liste des places
- Image de la zone ou de la place
- Nom de la place ou zone
- Catégorie
- Prix horaire
- Prix journalier
- Disponibilité
- Badge : disponible, occupé, réservé, maintenance
- Filtre par catégorie
- Filtre par prix

- Filtre par disponibilité
- Recherche par nom ou code
- Tri par prix croissant/décroissant
- Pagination ou infinite scroll

Exemple de catégories :

- Parking standard
 - Parking VIP
 - Parking couvert
 - Parking moto
 - Parking camion
 - Parking longue durée
 - Parking événementiel
-

5.3 Page détail d'une place

Chaque place doit avoir une page détail.

Informations à afficher :

- Images
- Nom de la place
- Code unique
- Catégorie
- Description
- Localisation interne
- Prix par heure
- Prix par jour
- Disponibilité en temps réel
- Services inclus
- Conditions de réservation
- Bouton "Ajouter au panier"
- Bouton "Réserver maintenant"

Services possibles :

- Caméra de surveillance
 - Gardiennage
 - Couverture
 - Recharge électrique
 - Nettoyage
 - Accès 24h/24
 - Badge sécurisé
-

5.4 Panier

Le panier doit permettre à l'utilisateur de préparer sa réservation.

Fonctionnalités :

- Ajouter une ou plusieurs réservations
- Choisir date et heure d'arrivée
- Choisir date et heure de sortie
- Calcul automatique de la durée
- Calcul automatique du montant
- Supprimer un élément du panier
- Modifier les horaires
- Afficher le total
- Appliquer un code promo
- Bouton checkout

Règle importante :

Une place ne peut pas être réservée deux fois sur le même créneau horaire.

5.5 Authentification

Pages obligatoires :

- Connexion
- Inscription
- Mot de passe oublié
- Profil utilisateur

Champs inscription :

- Nom
- Prénom
- Email
- Téléphone
- Mot de passe
- Confirmation mot de passe

Rôles :

- `client`
- `admin`

Un simple utilisateur inscrit doit avoir le rôle `client` par défaut.

5.6 Espace client

L'utilisateur connecté doit pouvoir :

- Voir ses réservations
- Voir les détails d'une réservation
- Annuler une réservation si elle n'est pas encore commencée

- Télécharger ou afficher un reçu
 - Modifier son profil
 - Voir l'historique de ses paiements
 - Contacter le support
-

5.7 Page contact

Le formulaire de contact doit contenir :

- Nom complet
- Email
- Téléphone
- Sujet
- Message

Après soumission :

- Enregistrer le message dans la base de données
 - Afficher un message de succès
 - Permettre à l'admin de consulter les messages
-

6. Paiement sandbox

Le module paiement doit permettre de simuler un paiement.

Workflow attendu :

1. L'utilisateur ajoute une réservation au panier
2. Il vérifie le résumé
3. Il clique sur "Payer"
4. Il est redirigé vers un paiement sandbox ou un écran de simulation
5. Si paiement réussi :
 6. créer la réservation
 7. créer la transaction
 8. vider le panier
 9. envoyer ou afficher une confirmation
10. Si paiement échoué :
 11. conserver le panier
 12. afficher une erreur

Statuts de paiement :

- pending
- success
- failed
- refunded

7. Back-office admin

L'admin doit avoir une interface séparée protégée par authentification et rôle.

URL recommandée :

/admin

7.1 Dashboard admin

Le dashboard doit afficher :

- Total des réservations
- Réservations du jour
- Revenus du jour
- Revenus du mois
- Nombre de clients
- Places disponibles
- Places occupées
- Taux d'occupation
- Dernières réservations
- Derniers paiements
- Graphique des revenus
- Graphique des réservations par catégorie

Les données importantes doivent être mises à jour en temps réel si possible.

7.2 Gestion des places

L'admin doit pouvoir :

- Ajouter une place
- Modifier une place
- Supprimer une place
- Changer le statut d'une place
- Ajouter des images
- Associer une catégorie
- Définir les prix
- Définir les services inclus

Champs d'une place :

- Nom
 - Code
 - Catégorie
 - Description
 - Image principale
 - Galerie d'images
 - Prix horaire
 - Prix journalier
 - Statut
 - Services
 - Localisation interne
 - Capacité si zone collective
 - Date de création
 - Date de modification
-

7.3 Gestion des catégories

L'admin doit pouvoir :

- Créer une catégorie
 - Modifier une catégorie
 - Supprimer une catégorie
 - Définir une icône
 - Définir une description
 - Définir l'ordre d'affichage
-

7.4 Gestion des réservations

L'admin doit pouvoir :

- Voir toutes les réservations
- Filtrer par statut
- Filtrer par date
- Filtrer par client
- Voir les détails
- Confirmer une réservation
- Annuler une réservation
- Marquer une réservation comme terminée

Statuts réservation :

- pending
- confirmed
- active
- completed
- cancelled

- expired
-

7.5 Gestion des paiements

L'admin doit pouvoir :

- Voir les transactions
 - Voir le statut du paiement
 - Voir le montant
 - Voir le client
 - Voir la réservation associée
 - Filtrer par période
 - Exporter les données en CSV
-

7.6 Gestion des utilisateurs

L'admin doit pouvoir :

- Voir les utilisateurs
 - Voir les détails d'un utilisateur
 - Activer ou désactiver un compte
 - Changer le rôle d'un utilisateur
 - Voir l'historique des réservations d'un utilisateur
-

7.7 Gestion des messages de contact

L'admin doit pouvoir :

- Lire les messages reçus
- Marquer un message comme lu
- Marquer un message comme traité
- Supprimer un message

Statuts message :

- new
 - read
 - resolved
-

7.8 Paramètres du site

L'admin doit pouvoir configurer :

- Nom du parking

- Logo
 - Email de contact
 - Téléphone
 - Adresse
 - Horaires d'ouverture
 - Devise
 - Prix par défaut
 - Liens réseaux sociaux
 - Texte de la landing page
 - Activation/désactivation du paiement sandbox
 - Mode maintenance
-

8. Entités principales à respecter

8.1 User / Profile

Représente un utilisateur de la plateforme.

Champs :

```
{
  id: string,
  firstName: string,
  lastName: string,
  fullName: string,
  email: string,
  phone: string,
  role: "client" | "admin",
  avatarUrl: string | null,
  isActive: boolean,
  createdAt: Date,
  updatedAt: Date
}
```

Règles :

- Un utilisateur inscrit est `client` par défaut.
 - Seul un admin peut modifier un rôle.
 - Un compte désactivé ne peut pas réserver.
-

8.2 ParkingCategory

Représente une catégorie de parking.

```
{
  id: string,
  name: string,
  slug: string,
  description: string,
  icon: string | null,
  imageUrl: string | null,
  displayOrder: number,
  isActive: boolean,
  createdAt: Date,
  updatedAt: Date
}
```

Exemples :

- standard
- vip
- covered
- motorcycle
- truck
- long-term

8.3 ParkingSpot

Représente une place ou une zone de stationnement.

```
{
  id: string,
  categoryId: string,
  name: string,
  code: string,
  slug: string,
  description: string,
  locationLabel: string,
  mainImageUrl: string,
  gallery: string[],
  hourlyPrice: number,
  dailyPrice: number,
  status: "available" | "occupied" | "reserved" | "maintenance",
  services: string[],
  capacity: number,
  isFeatured: boolean,
  isActive: boolean,
  createdAt: Date,
  updatedAt: Date
}
```

Règles :

- Le code doit être unique.
- Une place inactive ne doit pas apparaître côté client.
- Une place en maintenance ne peut pas être réservée.
- Le prix doit être supérieur à 0.

8.4 CartItem

Représente un élément du panier.

```
{
  id: string,
  userId: string,
  parkingSpotId: string,
  startDateTime: Date,
  endDateTime: Date,
  durationInHours: number,
  unitPrice: number,
  totalPrice: number,
  createdAt: Date,
  updatedAt: Date
}
```

Règles :

- `endDateTime` doit être supérieur à `startDateTime`.
- Le total est calculé automatiquement.
- Le panier peut être stocké dans Pinia, localStorage ou backend.

8.5 Reservation

Représente une réservation confirmée ou en attente.

```
{
  id: string,
  reservationNumber: string,
  userId: string,
  parkingSpotId: string,
  startDateTime: Date,
  endDateTime: Date,
  durationInHours: number,
  amount: number,
  status: "pending" | "confirmed" | "active" | "completed" | "cancelled" |
  "expired",
  paymentStatus: "pending" | "success" | "failed" | "refunded",
}
```

```
    createdAt: Date,  
    updatedAt: Date  
  }
```

Règles :

- Le numéro de réservation doit être unique.
- Une réservation confirmée bloque la place sur le créneau choisi.
- Une réservation peut être annulée seulement avant son démarrage.
- Une réservation terminée ne peut plus être modifiée.

8.6 Payment

Représente une transaction de paiement.

```
{  
  id: string,  
  reservationId: string,  
  userId: string,  
  provider: "stripe" | "paystack" | "flutterwave" | "manual",  
  providerReference: string,  
  amount: number,  
  currency: string,  
  status: "pending" | "success" | "failed" | "refunded",  
  paidAt: Date | null,  
  createdAt: Date,  
  updatedAt: Date  
}
```

Règles :

- Une réservation confirmée doit avoir un paiement réussi.
- Un paiement échoué ne doit pas confirmer la réservation.
- Le montant payé doit correspondre au montant de la réservation.

8.7 ContactMessage

Représente un message envoyé depuis le formulaire de contact.

```
{  
  id: string,  
  fullName: string,  
  email: string,  
  phone: string | null,  
  subject: string,  
}
```

```
message: string,  
status: "new" | "read" | "resolved",  
createdAt: Date,  
updatedAt: Date  
}
```

8.8 SiteSettings

Représente les paramètres globaux du site.

```
{  
  id: string,  
  siteName: string,  
  logoUrl: string | null,  
  contactEmail: string,  
  contactPhone: string,  
  address: string,  
  currency: string,  
  openingHours: string,  
  facebookUrl: string | null,  
  instagramUrl: string | null,  
  linkedinUrl: string | null,  
  sandboxPaymentEnabled: boolean,  
  maintenanceMode: boolean,  
  updatedAt: Date  
}
```

8.9 PromoCode

Représente un code promotionnel.

```
{  
  id: string,  
  code: string,  
  type: "percentage" | "fixed",  
  value: number,  
  maxUses: number,  
  usedCount: number,  
  startsAt: Date,  
  expiresAt: Date,  
  isActive: boolean,  
  createdAt: Date,  
  updatedAt: Date  
}
```

Règles :

- Un code expiré ne peut pas être utilisé.
 - Un code désactivé ne peut pas être utilisé.
 - `usedCount` ne doit pas dépasser `maxUses` .
-

8.10 AuditLog

Représente les actions importantes effectuées dans l'admin.

```
{
  id: string,
  userId: string,
  action: string,
  entity: string,
  entityId: string,
  description: string,
  createdAt: Date
}
```

Exemples d'actions :

- création d'une place
 - modification d'un prix
 - suppression d'une catégorie
 - annulation d'une réservation
 - changement de rôle utilisateur
-

9. Gestion d'état avec Pinia

Stores obligatoires :

authStore

Responsabilités :

- utilisateur connecté
- rôle
- login
- register
- logout
- refresh session
- vérification des permissions

parkingStore

Responsabilités :

- liste des places
- détails d'une place
- filtres
- recherche
- catégories
- chargement realtime

cartStore

Responsabilités :

- panier
- ajout au panier
- suppression
- calcul total
- code promo
- persistance localStorage

reservationStore

Responsabilités :

- création réservation
- liste des réservations client
- liste des réservations admin
- changement de statut

adminStore

Responsabilités :

- statistiques dashboard
- ventes
- revenus
- occupation parking
- messages contact
- paramètres du site

10. Lifecycles VueJS à pratiquer

Les stagiaires doivent utiliser explicitement les lifecycles dans plusieurs pages.

Exemples attendus :

onMounted

- charger les places au montage de la galerie
- charger les statistiques admin
- charger les détails d'une place

watch

- surveiller les filtres
- surveiller la recherche
- recalculer le total du panier
- surveiller le changement de catégorie

watchEffect

- déclencher automatiquement le refresh des données lorsque certaines dépendances changent

onUnmounted

- arrêter les listeners realtime Supabase/Firebase
- nettoyer les timers
- annuler les subscriptions

computed

- total panier
- nombre de places disponibles
- réservations filtrées
- revenus mensuels

11. Structure recommandée du projet

```
src/  
  assets/  
  components/  
    common/  
    layout/  
    parking/  
    cart/  
    admin/  
    forms/  
  composables/  
    useAuth.js  
    useParking.js  
    useCart.js  
    usePayment.js  
    useRealtime.js
```

```
layouts/  
  PublicLayout.vue  
  AuthLayout.vue  
  AdminLayout.vue  
pages/  
  public/  
    HomePage.vue  
    ParkingGalleryPage.vue  
    ParkingDetailPage.vue  
    ContactPage.vue  
  auth/  
    LoginPage.vue  
    RegisterPage.vue  
    ForgotPasswordPage.vue  
  client/  
    ProfilePage.vue  
    MyReservationsPage.vue  
    ReservationDetailPage.vue  
    CheckoutPage.vue  
  admin/  
    DashboardPage.vue  
    ParkingSpotsPage.vue  
    ParkingFormPage.vue  
    CategoriesPage.vue  
    ReservationsPage.vue  
    PaymentsPage.vue  
    UsersPage.vue  
    MessagesPage.vue  
    SettingsPage.vue  
router/  
  index.js  
  guards.js  
services/  
  supabaseClient.js  
  firebaseClient.js  
  parkingService.js  
  reservationService.js  
  paymentService.js  
  uploadService.js  
stores/  
  authStore.js  
  parkingStore.js  
  cartStore.js  
  reservationStore.js  
  adminStore.js  
utils/  
  formatCurrency.js  
  formatDate.js  
  calculateDuration.js  
  validators.js
```

App.vue
main.js

12. Routes obligatoires

```
[
  { path: "/", name: "home" },
  { path: "/parking", name: "parking.gallery" },
  { path: "/parking/:slug", name: "parking.detail" },
  { path: "/contact", name: "contact" },

  { path: "/login", name: "auth.login" },
  { path: "/register", name: "auth.register" },
  { path: "/forgot-password", name: "auth.forgot-password" },

  { path: "/checkout", name: "client.checkout" },
  { path: "/profile", name: "client.profile" },
  { path: "/my-reservations", name: "client.reservations" },
  { path: "/my-reservations/:id", name: "client.reservation.detail" },

  { path: "/admin", name: "admin.dashboard" },
  { path: "/admin/parking-spots", name: "admin.parking-spots" },
  { path: "/admin/categories", name: "admin.categories" },
  { path: "/admin/reservations", name: "admin.reservations" },
  { path: "/admin/payments", name: "admin.payments" },
  { path: "/admin/users", name: "admin.users" },
  { path: "/admin/messages", name: "admin.messages" },
  { path: "/admin/settings", name: "admin.settings" }
]
```

13. Règles de protection des routes

Routes publiques

Accessibles à tous :

- /
- /parking
- /parking/:slug
- /contact
- /login
- /register

Routes client

Accessibles uniquement aux utilisateurs connectés :

- `/checkout`
- `/profile`
- `/my-reservations`

Routes admin

Accessibles uniquement aux admins :

- `/admin`
- `/admin/*`

Si un simple client tente d'aller dans `/admin`, il doit être redirigé vers `/`.

14. Uniformisation attendue

14.1 Nommage des fichiers

Composants :

```
ParkingCard.vue
ParkingFilter.vue
ReservationTable.vue
AdminSidebar.vue
PaymentStatusBadge.vue
```

Stores :

```
authStore.js
cartStore.js
parkingStore.js
reservationStore.js
adminStore.js
```

Services :

```
parkingService.js
reservationService.js
paymentService.js
```

14.2 Nommage des variables

Utiliser le camelCase :

```
parkingSpot  
parkingSpots  
selectedCategory  
hourlyPrice  
dailyPrice  
isLoading  
errorMessage
```

14.3 Nommage des routes

Utiliser une logique claire :

```
admin.dashboard  
admin.parking-spots  
client.reservations  
parking.detail
```

14.4 Statuts uniformes

Tous les statuts doivent être en anglais et en minuscules :

```
available  
reserved  
occupied  
maintenance  
pending  
confirmed  
success  
failed  
cancelled  
completed
```

14.5 Format des prix

Tous les prix doivent être stockés en nombre.

Exemple :

```
hourlyPrice: 500  
dailyPrice: 5000
```

L'affichage doit être formaté côté interface :

500 FCFA
5 000 FCFA

14.6 Format des dates

Stockage :

ISO Date

Affichage :

29/06/2026 à 14:30

15. Contraintes UX/UI

Design

- Interface moderne
- Couleurs cohérentes
- Boutons visibles
- Cartes bien espacées
- Icônes claires
- Loader pendant les chargements
- Skeleton loader recommandé
- Toast de succès ou erreur
- Empty state lorsqu'il n'y a pas de données
- Modal de confirmation avant suppression

Mobile-first

Les pages doivent d'abord être pensées pour mobile.

Exigences :

- Menu mobile
 - Sidebar admin responsive
 - Cartes empilées sur mobile
 - Tables transformées en cards sur mobile
 - Boutons assez grands
 - Formulaires lisibles
 - Pas de scroll horizontal inutile
-

16. Contraintes fonctionnelles importantes

Réservation

- Un utilisateur doit être connecté pour réserver.
- Une place indisponible ne peut pas être réservée.
- Une réservation ne peut pas avoir une date de fin inférieure à la date de début.
- Une réservation confirmée doit apparaître dans l'espace client.
- L'admin doit voir toutes les réservations.

Paie ment

- Le paiement doit passer par un mode sandbox ou simulation.
- Une réservation n'est confirmée que si le paiement est réussi.
- Un paiement échoué ne doit pas vider le panier.

Admin

- Seul l'admin peut accéder au back-office.
- Toute suppression importante doit demander confirmation.
- Les actions importantes doivent être enregistrées dans `AuditLog`.

Realtime

Les éléments suivants doivent être actualisés en temps réel si possible :

- disponibilité des places
- nouvelles réservations
- paiements
- statistiques dashboard
- messages de contact

17. Contraintes de sécurité

- Ne pas exposer les clés secrètes côté frontend.
 - Protéger les routes admin.
 - Vérifier les rôles côté backend ou règles de sécurité.
 - Empêcher un utilisateur de voir les réservations d'un autre utilisateur.
 - Empêcher un client de modifier les prix.
 - Valider les données avant insertion.
 - Sécuriser les uploads d'images.
 - Limiter les droits d'écriture côté client.
-

18. Livrables attendus

Chaque stagiaire ou groupe doit fournir :

1. Code source complet
 2. README clair
 3. Lien de démo déployée
 4. Accès test client
 5. Accès test admin
 6. Captures d'écran
 7. Diagramme simple des entités
 8. Description des choix techniques
 9. Liste des fonctionnalités terminées
 10. Liste des limites ou améliorations possibles
-

19. README attendu

Le README doit contenir :

- Nom du projet
 - Description
 - Fonctionnalités principales
 - Stack utilisée
 - Installation
 - Configuration `.env`
 - Lancement du projet
 - Comptes de test
 - Structure du projet
 - Captures d'écran
 - Difficultés rencontrées
 - Améliorations futures
-

20. Critères d'évaluation

Frontend – 25 points

- Qualité de l'interface : 8 pts
- Responsive design : 5 pts
- Composants réutilisables : 5 pts
- Animations et interactions : 3 pts
- Qualité des formulaires : 4 pts

VueJS – 25 points

- Composition API : 5 pts

- Vue Router : 4 pts
- Pinia : 5 pts
- Lifecycles : 5 pts
- Composables : 3 pts
- Guards de routes : 3 pts

Backend – 20 points

- Authentification : 4 pts
- CRUD : 5 pts
- Upload images : 3 pts
- Realtime : 4 pts
- Règles de sécurité : 4 pts

Fonctionnalités métier – 20 points

- Galerie parking : 4 pts
- Filtres/recherche : 3 pts
- Panier : 4 pts
- Réservation : 4 pts
- Paiement sandbox : 3 pts
- Dashboard admin : 2 pts

Qualité professionnelle – 10 points

- README : 2 pts
- Organisation du code : 3 pts
- Nommage propre : 2 pts
- Gestion des erreurs : 2 pts
- Déploiement : 1 pt

Total : **100 points**

21. Bonus possibles

Les stagiaires peuvent gagner des points bonus s'ils ajoutent :

- Notifications email
- QR code pour réservation
- Export PDF du reçu
- Export CSV des ventes
- Mode sombre
- Carte interactive du parking
- Gestion des abonnements mensuels
- Historique des actions admin
- Tests unitaires
- PWA installable
- Notifications push
- Chat support client/admin

- Reconnaissance automatique de plaque fictive
 - Statistiques avancées
 - Multi-langue français/anglais
-

22. Niveau minimal attendu

Pour valider le projet, il faut au minimum :

- Landing page responsive
 - Authentification
 - Liste des places
 - Détail d'une place
 - Filtres
 - Panier
 - Réservation
 - Paiement simulé
 - Espace client
 - Admin CRUD places
 - Dashboard simple
 - Backend connecté
 - Code propre et structuré
-

23. Niveau excellent attendu

Un très bon projet doit avoir :

- Interface très professionnelle
- Realtime fonctionnel
- Dashboard dynamique
- Paiement sandbox bien intégré
- Sécurité backend propre
- Upload images
- Rôles bien gérés
- Filtres avancés
- Panier persistant
- Code modulaire
- Composables réutilisables
- README complet
- Déploiement fonctionnel